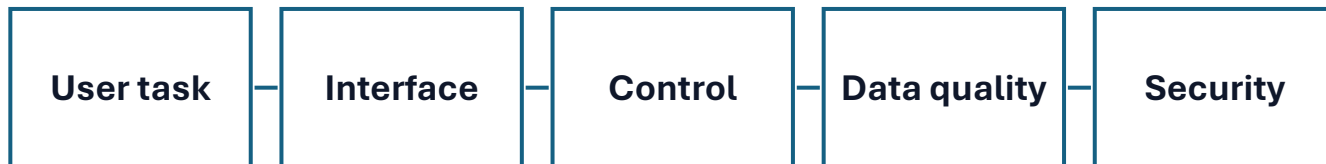


TOPIC 4

Revised into two teaching sessions



UI/UX,
Forms,
Reports
Interfaces,
Controls &
Security

Lesson Structure

Session 1 **UI/UX, Forms, Reports**

Form • Report • Usability • Validation



Session 2 **Interfaces, Controls & Security**

Dialogue • Access control • Audit trail •
Security

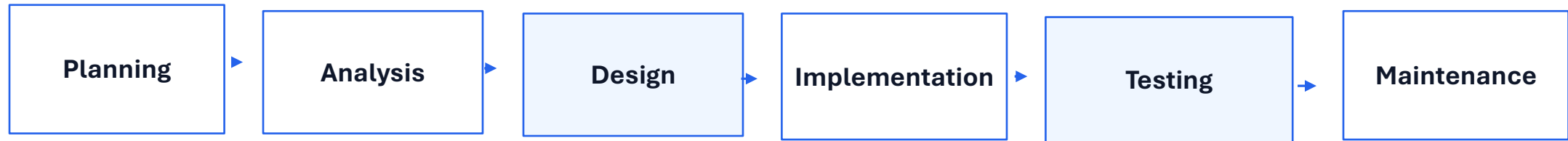
Goal: students can design user-facing screens and explain the controls behind them.

Learning Outcomes

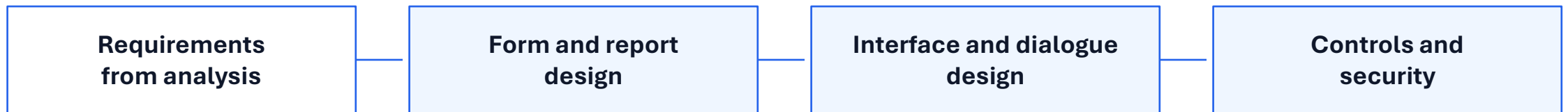
- Distinguish UI, UX, forms, reports, interfaces, and dialogues.
- Apply user-centered design to form and report prototypes.
- Select suitable input controls and validation rules.
- Design readable reports for operational and managerial decisions.
- Explain interface and dialogue design deliverables.
- Design a simple navigation/dialogue sequence.
- Identify input, processing, output, file, and access controls.
- Apply basic security principles to an information system interface.

Expected output: students can design a form, report, interface flow, and basic control plan for one scenario.

How Week 07 Fits into the SDLC



Week 07 focuses on design outputs users can see and controls users may not see.



Design logic

Analysis says what the system must do. Design decides how users interact with it, how data is entered and presented, and how system behavior is controlled.

Two-Session Teaching Plan

Session 1: UI/UX, Forms, Reports

Focus: user goals, form/report content, layout, validation, feedback, usability, and prototyping.

Activity: design a request form and a pending approval report.

Session 2: Interfaces, Controls & Security

Focus: interface flow, dialogue, control types, access control, audit trail, and secure approval workflow.

Activity: design an approval interface flow and control plan.

Session 1

UI/UX, Forms, Reports

A solid orange horizontal bar spanning the width of the slide content area, positioned below the main title.

Design what users see, enter, read, and use for decisions.

Session 1 Overview:

UI/UX, Forms and Reports

After completing this session, students will be able to:

- Explain basic UI/UX concepts in information system design.
- Design user-centered forms for data entry and task completion.
- Select suitable input controls, validation rules, and feedback messages.
- Design readable reports that support operational or managerial decisions.
- Apply usability principles to evaluate form and report prototypes.
- Justify form and report design decisions based on user needs and data quality.

UI and UX: Definitions

UI = what users see and interact with

UI includes

Screens, buttons, labels, menus.
Fields, icons, messages.
Forms, tables, report layouts.
Navigation and visual hierarchy.

UX = how users experience completing a task

UX includes

Clarity and confidence.
Speed and task completion.
Error recovery and learnability.
Satisfaction and acceptance.

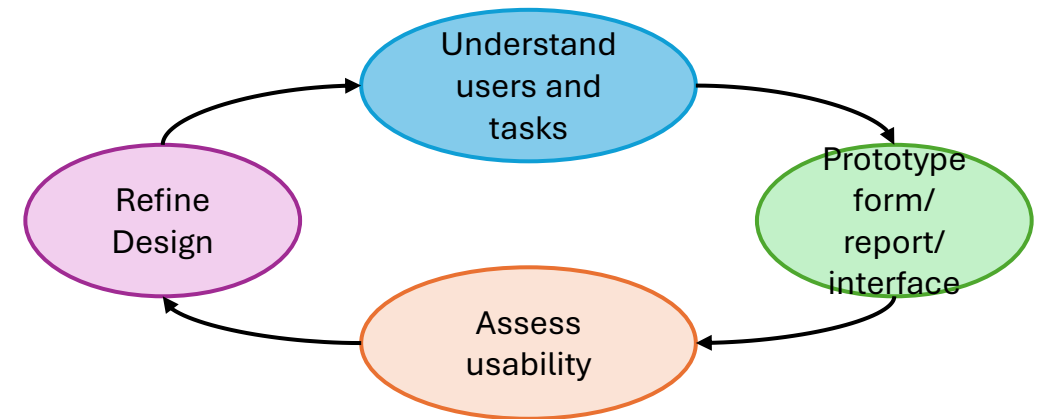
Key idea: an attractive UI can still have poor UX if users cannot complete the task correctly.

Forms, Reports, Interfaces, and Dialogues

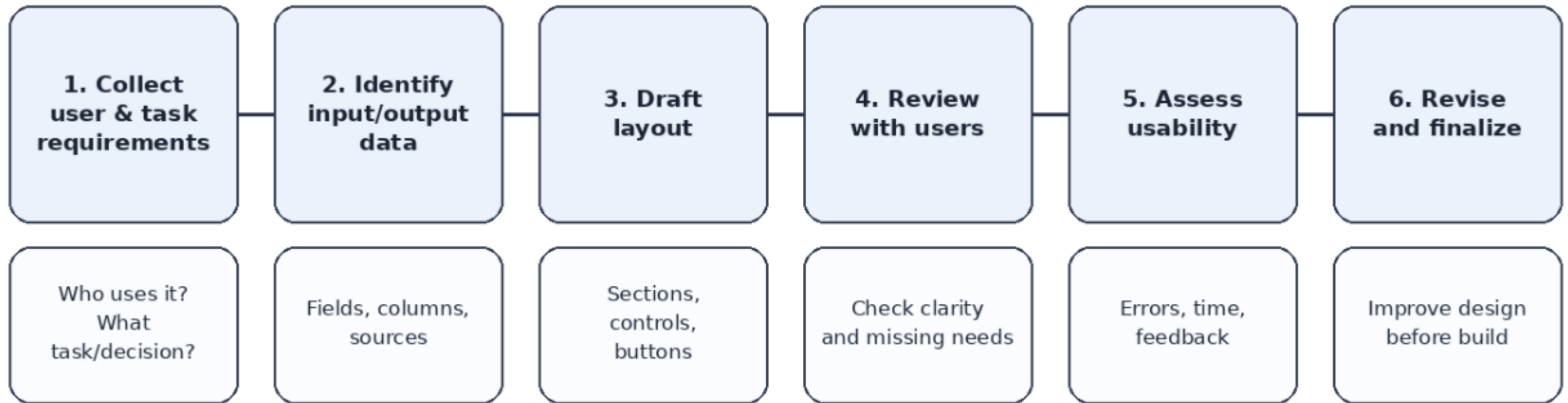
- **Form:** A business document or screen used to collect or present data, often for input or editing.
- **Report:** A business document or screen used to present information for reading, monitoring, or decision-making.
- **Interface:** The method by which users interact with the information system.
- **Dialogue:** The sequence of interactions between the user and the system.

Example

Create Booking Form → Confirmation Screen →
Booking Status Page → Monthly Booking Report



Form and Report Design Process



Main deliverables:

- Narrative overview
- Sample design
- Field definitions
- Testing and usability notes

Exam Answer Structure: Form and Report Design

When answering a form/report design question, students should include:

Part	What to include
Purpose and user	Who uses it? What task or decision does it support?
Sample layout	Main sections, fields, buttons, report columns.
Field / column definitions	Name, type, required/optional, source.
Controls / validation	Dropdown, date picker, required check, business rule.
Usability explanation	Clear labels, grouping, readability, feedback, error handling.

Why UI/UX Matters in Systems Analysis and Design

Good UI/UX design helps to:

- Reduce user errors.
- Improve task completion speed.
- Increase user acceptance.
- Improve data quality.
- Reduce training and support needs.
- Make reports easier to interpret.

Example

A booking form with clear validation prevents invalid time ranges before the data reaches the database.

User-Centered Design

User-centered design means designing from the perspective of real users and real tasks.

Key questions

- Who will use the form or report?
- What task are they trying to complete?
- What information do they need?
- When and where will they use it?
- What errors are likely?
- What feedback will help them recover?

Example

Student user: submit a booking request.

Facility officer: review pending requests and detect conflicts.

The design must support both users differently

Principles of Good Form Design

A good form should:

- Match the user's task sequence
- Use clear and familiar labels
- Group related fields
- Show required fields clearly
- Minimize unnecessary typing
- Use suitable input controls
- Provide examples or input hints
- Validate data before submission

Form Layout and Grouping

Requester Information

- Name
- Email
- Role

Room and Time

- Building
- Room
- Date
- Time

Purpose and Equipment

- Purpose
- Participants
- Equipment

Actions

- Save Draft
- Submit
- Cancel

Good layout places related information together, uses headings and spacing, and separates primary actions from secondary actions.

Interface and Dialogue Design

- Interface = visible interaction layer
- Dialogue = interaction sequence
- Forms and reports are part of the interface

Example dialogue

Login → Dashboard → Create Booking → Review Details → Submit → Confirmation

Good forms must fit naturally into the user interaction flow.

Interface Design Supports User Tasks

Design Aspect	Example
Navigation	Dashboard → Create Booking
Layout	Group room/date fields together
Consistency	Same Submit button style
Feedback	Success + error messages
Simplicity	Avoid unnecessary fields

In Session 1, interface design focuses on supporting form/report usability.

Common Form Design Mistakes

1. Too many fields

✗ Bad

First Name

Middle Name

Last Name

Email Address

Phone Number

Department

Faculty

Building

Room Number

Purpose

Event Type

Expected Attendees

Additional Notes

✓ Good

Requestor

Email Phone

Department Room *

Date *

Time *

Purpose *

Expected Attendees

2. Unclear labels

✗ Bad

Name

Date

Time

Rm

Fee

Notes

✓ Good

Full Name

Booking Date

Start Time

Room Number

Estimated Cost

Additional Notes

3. No validation

Date	Date
32/13/2026	20/05/2026
Email	Email
abc@	student@abc.edu
Submit	

Common Form Design Mistakes

4. Poor grouping

✗ Bad

Requestor
Name
Email
Phone

Room
Date
Time
Purpose
Notes

Submit

✓ Good

Requestor Information
Name
Email
Phone

Booking Details
Room
Date
Time
Purpose
Additional Notes

Submit

5. Overuse of text boxes

✗ Bad

Requestment

Room Number

Purpose

Event Type

Submit

✓ Good

Department

Room Number

Purpose

Event Type

Submit

6. Weak feedback

✗ Bad

Error!
Please check your information.

OK

✓ Good

Email
Email must be in correct format.

Date
Date is invalid.

Update

Good vs Poor Report Design

Poor Design	Improved Design
Too many columns	Important columns only
No grouping	Logical grouping
No filtering	Search/filter
Technical names	Business-friendly labels

Form Design and Data Quality

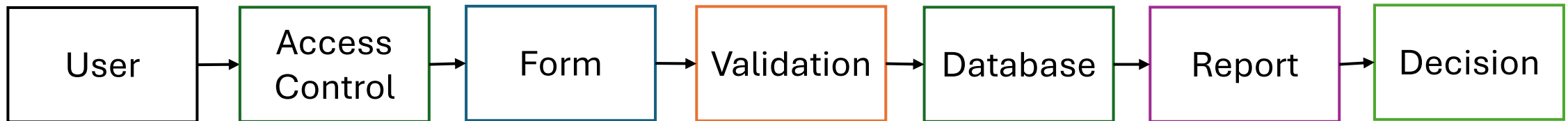
- UI design affects database quality
- Validation reduces inconsistent data
- Controlled inputs improve reporting accuracy

Example:

Input room name

Choose room ▼

Relationship Between UI, UX, Validation, and Security



Interface problems become data quality and security problems.

Data Entry Design Principles

- Use default values.
- Use dropdowns for controlled values.
- Minimize typing.
- Prevent errors before submission.
- Allow users to review before final submit.

Navigation Design

Navigation design should help users know:

- Where they are.
- What they can do next.
- Confirmation status.

Example navigation structure

Dashboard → My Bookings → Create Booking → Booking Details → Confirmation

Form Design from User Task Perspective

User Task	Design Decision
Select room quickly	Use dropdown instead of text input
Avoid invalid booking	Use date/time validation
Reduce typing	Auto-fill requester information
Prevent confusion	Group fields into sections
Confirm submission	Show success message + Booking ID

Good form design translates user tasks into interface decisions.

Common Input Controls

Choose controls based on the type of data.

Data need	Suitable control	Example
Open text	Text box / Text area	Purpose description
One value from many options	Dropdown list	Room, building, status
One value from few options	Radio buttons	Student / staff role
Multiple choices	Checkboxes	Equipment needed
Date/ Time	Date/ Time picker	Booking date and time
Existing record	Lookup field	Doctor, room, student account

Validation Flow

Input controls improve data quality before data is saved.



Common Validation Rules

Common validation rules

Validation type	Example
Required check	Room, date, start time, end time, and purpose cannot be blank.
Format check	Email must follow a valid email pattern.
Range check	Participant count must be greater than 0.
Business rule	End time must be later than start time.
Availability check	Room must not already be booked for the selected time.
Duplicate check	Avoid repeated submissions for same requester, room, and time.

Error Messages and Feedback

Poor feedback

“Invalid input.”

Problem: vague, unhelpful, and gives no recovery step.

Rules:

- Immediate
- Clear
- Polite
- Specific
- Actionable
- Safe: no technical details or sensitive data

Better feedback

“The end time must be later than the start time. Please choose a valid time range.”

Strength: specific, polite, and actionable.

Help Design

System help can include:

- Field-level hints.
- Tooltips.
- Examples.
- Frequently asked questions.
- Guided steps.
- Context-sensitive help.

Example

Purpose field hint: “Briefly describe the academic or administrative reason for the booking.”

Report Design: Purpose and Audience

A report should answer a decision need — not display every available field.

Before designing a report, ask:

- Who will read it?
- What decision or task does it support?
- What level of detail is needed?
- How often is it produced?
- Should it be printed, exported, or viewed online?

Example

Facility officers need a pending booking report to decide which requests require review.

Common Report Types

Choose the report type based on the user's task.

Type	Purpose	Example
Detail	Shows transaction-level data	Daily booking list
Summar	Aggregates data by category or period	Monthly room utilization
Exception	Highlights unusual or problem cases	Overdue approvals
Status	Shows current state of work items	Pending requests
Analytical	Supports comparison or trend analysis	Utilization by building

Paper vs Electronic Reports

Type	Strength	Limitation
Paper/PDF	Formal, printable	Less interactive
Web report	Easy access	Needs good navigation
Dashboard	Visual summary	Can oversimplify
Export file	Further analysis	Risk of uncontrolled sharing

Formatting Reports

Good report formatting should include:

- Clear title.
- Date or reporting period.
- Logical grouping.
- Meaningful column names.
- Consistent alignment.
- Totals and subtotals when appropriate.
- Highlighted exceptions.
- Page number or export information if needed.

Formatting Reports, Tables, and Lists

When designing table-based screens or reports:

- Show only necessary columns.
- Use meaningful headings.
- Align text and numbers consistently.
- Provide sorting and filtering when useful.
- Avoid too many columns on one screen.
- Highlight exceptions without overdecorating
- Include title, period, date generated, and page/export information

BookingID	Requester	Room	Date	Time	Status
BKG-001	Nguyen A	B-204	18 May	09:00–11:00	Pending
BKG-002	Tran B	A-101	18 May	13:00–15:00	Approved
BKG-003	Le C	C-305	19 May	10:00–12:00	Conflict

Usability Success Factors

Usability = the design assists user performance instead of hindering it.

Effectiveness

Can users complete the task?

Efficiency

Can users complete it quickly?

Accuracy

Does the system produce what users expect?

Satisfaction

Do users feel comfortable and confident?

Learnability

Can new users understand it?

Error tolerance

Can users recover from mistakes?

A visually attractive form is not enough. Students should be able to explain how the design improves speed, accuracy, satisfaction, and recovery from errors.

Color, Highlighting, and Visual Emphasis

Use visual emphasis carefully:

- Use highlighting to draw attention to important information.
- Use color consistently and meaningfully.
- Do not rely only on color to communicate meaning.
- Avoid excessive decoration.
- Ensure readability and sufficient contrast.

Design warning

Highlighting loses power when everything is highlighted. Reserve strong emphasis for urgent, abnormal, or decision-critical information.

Approved

Pending review

Rejected

Overdue approval

Accessibility and Inclusive Design

Accessible design considers users with different abilities, devices, and levels of technical experience.

Basic practices

- Use readable font sizes and sufficient contrast.
- Do not rely only on color to communicate status.
- Use plain language and clear labels.
- Support keyboard navigation where possible.
- Avoid unclear icons without text labels.
- Write error messages that can be understood by non-technical users.

Status should use color + text
-> Approved, Pending, Rejected

Approved

Pending

Rejected

Prototype and Usability Review



Prototype fidelity increases as the design becomes clearer.

Prototype benefits

- Makes ideas visible.
- Supports user feedback.
- Reveals hidden requirements.
- Reduces redesign cost.

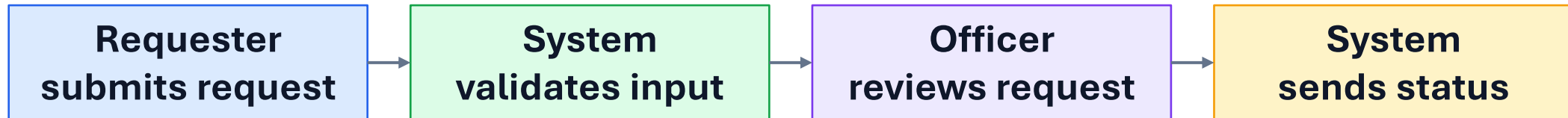
Review checklist

- Are fields understandable?
- Are required fields clear?
- Are errors easy to fix?
- Is navigation predictable?
- Does the report support a real decision?

Example Form: Facility Booking Request

Scenario

A university system allows students and staff to submit room booking requests. Each booking includes requester details, room, date, time, purpose, participant count, equipment requests, and status. A facility officer reviews pending requests and approves or rejects them.



We use this scenario for form, report, interface flow, controls, and security examples.

Example Form: Facility Booking Request

Form sections

1. Requester information

- Name, email, role.

2. Booking information

- Room, date, start time, end time.

3. Booking purpose

- Purpose, number of participants.

4. Equipment request

- Projector, microphone, speaker.

5. Actions

- Submit, Save Draft, Cancel.

Validation rules

- Room, date, start time, end time, and purpose are required.
- End time must be later than start time.
- Participants must not exceed room capacity.

Wireframe Example: Create Facility Booking Request

Facility Booking System

Dashboard My Bookings
Notifications Profile

Requester Information

Name (auto-filled)Email (auto-filled)

Room and Time

Building ▼Room ▼Date 

Start time End time Participants

Purpose and Equipment

Purpose / academic reason

☐ Projector ☐ Microphone ☐ Speaker

Save DraftSubmit

- Auto-fill known requester data.
- Use dropdowns for building and room.
- Use date/time pickers for scheduling.
- Show required fields clearly.
- Separate primary and secondary actions.

Example Validation and Feedback

Validation rules

- Booking date cannot be in the past.
- End time must be later than start time.
- Participant count must not exceed room capacity.
- Room must be available in the selected time range.
- Purpose is required before submission.

Feedback examples

Success: “Your booking request has been submitted. Booking ID: BKG-2026-015.”

Error: “Room B-204 is already booked from 09:00 to 11:00. Please choose another time or room.”

Feedback should explain result + next step.

Example Report:

Pending Booking Approval Report

Purpose Help facility officers review pending booking requests.

Suggested columns

- BookingID.
- Requester.
- Room.
- Date.
- Start time.
- End time.
- Participant count.
- Equipment requested.
- Submission date.
- Status.

Useful features

- Filter by date, room, building, and status.
- Sort by submission time.
- Highlight urgent requests.

Report Mockup: Pending Booking Approval Report

Pending Booking Approval Report				Period: 20–25 May 2026 Generated: 20 May 2026		
Filter: Building ▼		Filter: Status ▼		Search Requester		
BookingID	Requester	Room	Date	Time	Participants	Status
BKG-015	Nguyen A	B-204	18 May	09:00–11:00	35	Pending
BKG-016	Tran B	A-101	18 May	13:00–15:00	28	Pending
BKG-017	Le C	C-305	19 May	10:00–12:00	60	Conflict

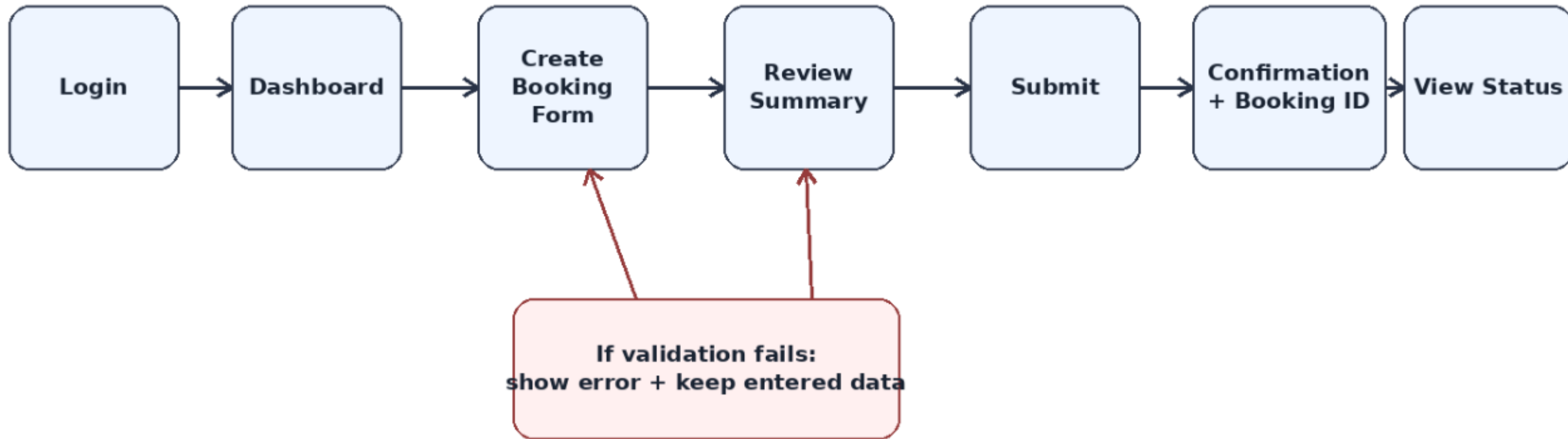
Example Dialogue: Submit a Booking Request

Dialogue sequence

1. User logs in.
2. User opens Dashboard.
3. User selects Create Booking.
4. System displays booking form.
5. User enters booking details.
6. System validates input.
7. User reviews summary.
8. User submits request.
9. System displays confirmation and BookingID.
10. User can view request status.

Example Dialogue: Submit a Booking Request

Users should always know where they are,
what happens next, and whether the action is
complete.



In-Class Activity 1: Form and Report Design

Using the University Facility Booking System

Task A: Form design

Design the Create Booking Form: sections, required fields, input controls

Task B: Report design

Design one report for facility officers: title, purpose, columns, sorting/filtering, and highlighting rules.

Task C: Validation and Feedback

Design Booking Form with sections, required fields, input controls, validation rules, and error messages.

Session 1 Conclusion

Key takeaways:

- UI is the visible interaction layer; UX is the overall experience.
- Forms should collect valid, complete, and necessary data.
- Reports should support specific decisions and user needs.
- Good forms use suitable controls, validation, feedback, and help.
- Good reports use clear purpose, meaningful columns, and readable formatting.
- A good answer explains layout + controls + validation + feedback + usability.

*Session 2 continues from “what users see” to
“how interaction is controlled and secured.”*

Session 2

Interfaces, Controls & Security

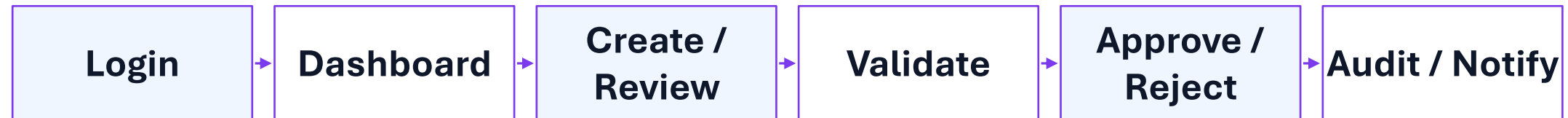


Design how users move, act safely, and access protected information.

From Forms/Reports to Interfaces/Controls

Session 1 asked: What should users see, enter, and read?

Session 2 asks: How should users move, act, recover, and be controlled?



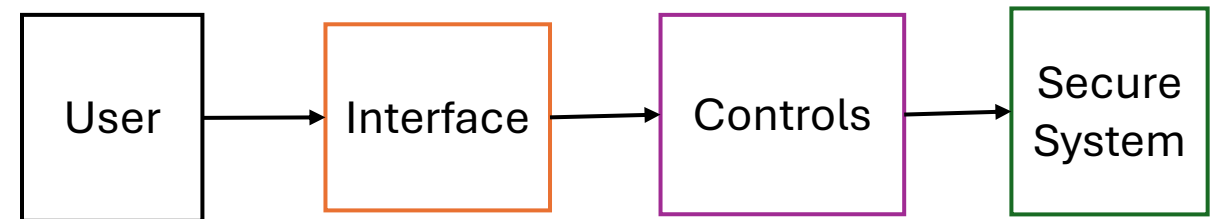
Interface design defines the conversation between user and system.

A form is one screen. An interface is the larger interaction environment. Controls and security define what actions are allowed, checked, recorded, and protected.

Session 2 Overview

This session covers:

- Interface and dialogue design
- Navigation, interaction methods, feedback, error handling, and help.
- Control types: input, processing, output, access, audit, backup/recovery
- Security basics: CIA, authentication, authorization, RBAC, least privilege
- Applied control plan for facility booking approval



What Is an Interface?

An interface is the point of interaction between users and the information system.

It includes:

- Screens
- Menus
- Buttons
- Forms
- Messages
- Navigation links
- Icons
- Input and output devices

Key idea:

The interface connects user goals with system functions.

Interface Design in ISAD

Interface design transforms requirements into user interaction.

Requirement example:

“Facility officers can approve or reject booking requests.”

Interface design output:

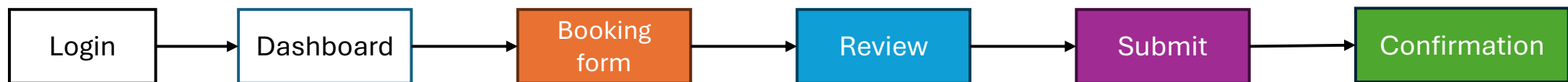
- Pending request list
- Request detail screen
- Approve button
- Reject button
- Comment field
- Confirmation message
- Updated status display

Dialogue Design

A dialogue is the sequence of interactions between the user and the system.

Example dialogue:

1. User logs in
2. System shows dashboard
3. User selects “Create Booking”
4. System displays booking form
5. User enters details
6. System validates input
7. User reviews summary
8. System confirms submission



Interface vs Dialogue

Interface

The visible interaction layer: screens, menus, fields, controls, labels, messages, and navigation.

Simple comparison:

Interface = visible interaction layer

Dialogue = interaction sequence

Example:

Login → Dashboard → Pending Requests → Request Detail → Approve/Reject → Confirmation

Dialogue

The sequence of interactions: how users move from one display to another and how the system responds.

Interface	Dialogue
Screens	Step 1
Buttons	Step 2
Fields	Step 3

Goals of Good Interface Design

A **good interface** should be:

- Clear
- Consistent
- Simple
- Responsive
- Forgiving
- Accessible
- Secure
- Task-oriented

Design principle:

Users should understand what to do, what happened, and what to do next.

Interaction, Navigation, and Layout

Topic	Design focus	Example
Interaction method	How users issue commands	Menu + form-based interaction for students/officers
Navigation	Where users are and what comes next	Dashboard → Create Booking → Review → Confirmation
Layout	How information is grouped and scanned	Requester Room and Time Purpose Actions
Consistency	Same pattern across screens	Use “Submit Request” consistently

Feedback, Error Handling, and Help

Users should know what happened and how to recover.

Feedback

**Immediate, clear, specific,
polite, actionable, safe.**

Error handling

**Identify the problem, show
where it occurred, preserve
entered data, and explain
how to fix it.**

Help

**Use hints, examples,
tooltips, guided steps, and
context-sensitive help.**

Good example:

“The selected room is unavailable at this time. Please choose another time or room.”

Interface Prototype

A prototype is an early sample of an interface or dialogue.

Prototype types:

- Paper sketch
- Wireframe
- Clickable mockup
- Simple working screen

Benefits:

- Makes ideas visible
- Supports user feedback
- Reveals hidden requirements
- Reduces redesign cost

Interface Layout Structure

Header: system name + user role

**Navigation
menu**

Main task area
Form / report / dashboard

Feedback / Help / Warnings

- Use consistent structure across screens.
- Place navigation where users expect it.
- Separate task content from system messages.
- Make role and permissions visible when useful.
- Keep destructive actions visually distinct and confirmed.

Controls in Information Systems

Controls are design features that help ensure the system operates correctly, securely, and reliably.

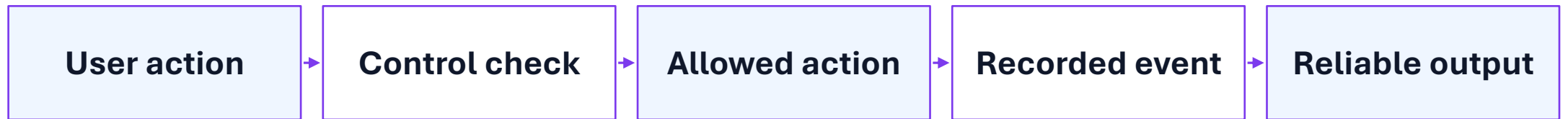
Main purposes:

- Prevent errors
- Detect problems
- Correct issues
- Protects assets

Example:

Validation rules, authorization checks, approval workflow, audit trail, backup, confirmation prompts, report access restrictions, and exception logs.

Why Controls Matter in Interface Design



- Protect data accuracy and completeness.
- Reduce invalid transactions before they are stored.
- Prevent unauthorized actions.
- Support accountability through audit logs.
- Increase user confidence and organizational trust.
- Reduce risk when reports are used for decisions.

**Controls must be designed
where risk appears.**

Types of Controls

Use control types as a checklist for design.

Control type	Purpose	Facility booking example
Input	Ensure entered data is valid and complete	Required fields, date/time validation, room dropdown
Processing	Ensure operations follow business rules	Only Pending requests can be approved/rejected
Output	Ensure displayed/exported information is appropriate	Approval report visible only to officer/admin
Access	Restrict screens, data, and actions	Student sees own requests; officer reviews requests
Audit	Record important actions	Log submit, approve, reject, cancel, export
Recovery	Restore service and data if problems occur	Backup booking, room, equipment, and audit data

Input Controls

Input controls ensure that entered data is valid, complete, and meaningful.

Field existence

Required data is present.

Data type

Only valid type is accepted.

Range / reasonableness

Value is within allowed limits.

Format

Pattern follows the expected format.

Validity

Value exists in reference data.

Combination check

Set of fields makes sense together.

Examples:

- Required fields
- Data type checks
- Format checks
- Range checks
- Dropdown lists

- Default values
- Duplicate checks
- Business rule validation

Facility booking example:

Booking date cannot be in the past.

Processing Controls

Processing controls ensure that system operations are performed correctly.

Examples:

- Status transition rules
- Calculation checks
- Approval workflow rules
- Conflict checking
- Transaction confirmation
- Error handling during processing

Example:

Submitted → Approved

Submitted → Rejected

Approved → Cancelled

Rejected cannot become Approved without resubmission.

Output Controls

Output controls ensure that reports and displayed information are accurate, complete, and appropriate.

Examples:

- Report title and date
- Correct filter criteria
- Summary totals
- Page numbering
- Export restrictions
- Sensitive data masking
- Access control for reports

Example:

Only facility officers can export the full pending booking report.

Access Controls and Role-Based Access Control

Access control determines who can do what.

Role	Main permission
Student	Create request, view own status
Staff	Create request, view own status
Facility Officer	Review, approve, reject, view approval reports
Admin	Manage users, rooms, roles, system settings

Authentication = Who are you?

Authorization = What can you do?

Least privilege = give only the minimum access needed.

Unauthorized actions should be hidden and blocked.

Example:

- Student → Create/View own bookings
- Officer → Review/Approve requests
- Admin → Manage users/rooms

Security Principles: CIA Triad

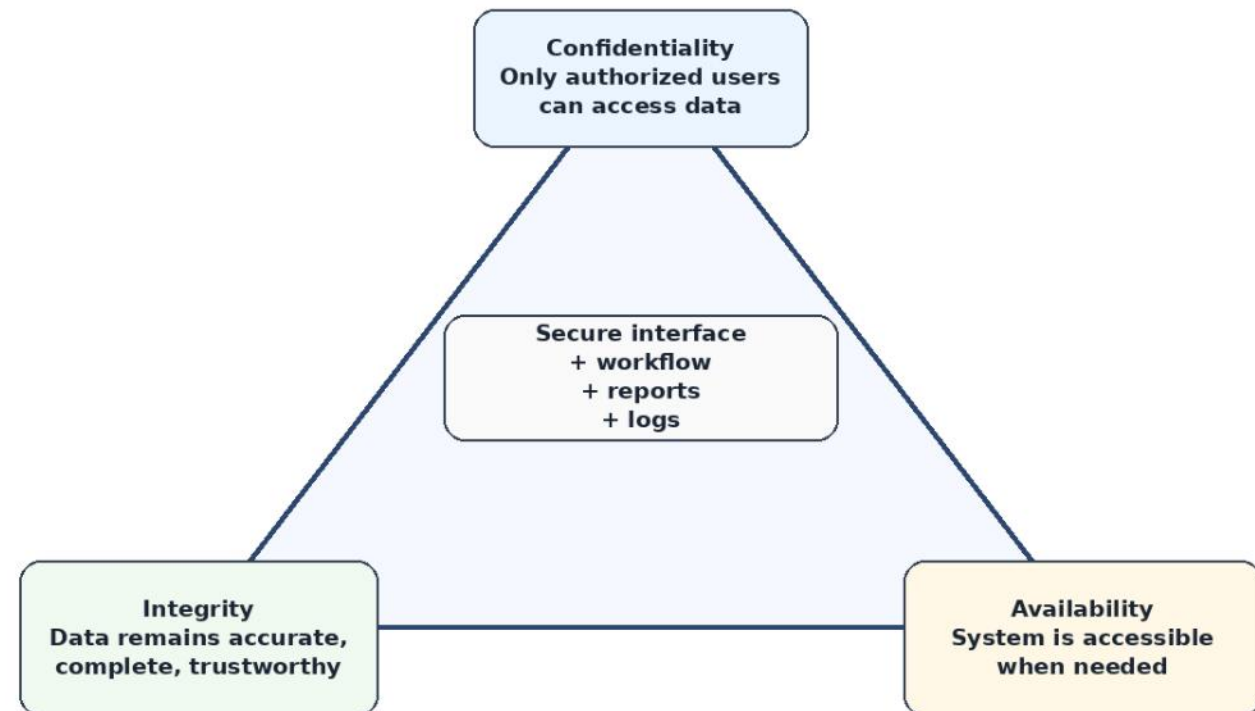
Security must be designed into screens, workflows, reports, and logs.

**Authentication → Authorization →
Allowed action**

Example:

Booking data should be private, correct, and available during working hours.

Commonly focuses on three principles:



Secure Interface Design

Secure interface design should:

- Hide unauthorized functions
- Disable invalid actions
- Confirm sensitive actions
- Avoid exposing technical error details
- Mask sensitive data when appropriate
- Use secure session timeout
- Log important actions

Example:

The Approve button is visible only to facility officers.

Audit Logs

An audit log records important system activities.

Common log data:

- User ID
- Action performed
- Date and time
- Record affected
- Previous value
- New value
- IP address or device information

Audit Log Example: Booking Approval

Log ID	User ID	Role	Booking ID	Action	Old status	New status	Timestamp	Comment
L001	OFF-03	Officer	BKG-015	Approve	Pending	Approved	20 May 2026 10:35	Valid request
L002	OFF-04	Officer	BKG-017	Reject	Pending	Rejected	20 May 2026 10:42	Room conflict
L003	ADM-01	Admin	ROOM-B204	Update	Capacity 35	Capacity 40	20 May 2026 11:05	Room updated

Audit logs support accountability.

They should record who did what, when, to which record, and what changed.

Data Privacy in Interface Design

Data privacy means protecting personal and sensitive information.

Interface practices:

- Show only necessary data
- Mask sensitive fields
- Limit export options
- Avoid unnecessary personal details
- Use clear consent or notice when needed
- Control report access

Example:

A report may show requester name but hide phone number unless necessary.

Interface, Controls, and Security Checklist

Before finalizing the design, check:

- Are navigation paths clear?
- Are labels consistent?
- Are errors easy to fix?
- Are input fields controlled?
- Are business rules validated?
- Are user roles clearly defined?
- Are unauthorized actions hidden or blocked?
- Are important actions logged?
- Are reports protected?
- Is sensitive data shown only when needed?

Applying Controls to Facility Booking

A good design explains both the screen and the control behind the screen.

Area	Example design decision
Input	Use room dropdown, required fields, date/time validation, availability check.
Processing	Only Pending requests can be approved/rejected; conflict check before approval.
Output	Approval report visible only to officer/admin; filter period shown on report.
Access	Student views own requests; officer reviews assigned requests; admin manages rooms.
Audit	Record submitter, reviewer, decision, timestamp, old/new status, and reason.
Recovery	Backup booking, room, equipment, and audit data; test restoration.

In-Class Activity 2:

Interface, Controls & Security

Scenario:

A facility officer reviews pending room booking requests. The officer can approve or reject a request. If rejecting, the officer must enter a rejection reason. The system must prevent unauthorized users from approving requests and must record all approval decisions.

Task A: Dialogue sequence

Draw the main screens and arrows for “approve or reject booking request.” Include success and error paths.

Task C: Interface Flow with and Security rationale

Draw the Approval Control flow and
Explain how your design protects confidentiality, integrity, and availability.

Task B: Control plan

List at least 2 input controls, 2 processing controls, 2 output controls, and 2 access/security controls.

Required condition

If rejecting, the officer must enter a rejection reason; all approval decisions must be recorded.

Final Conclusion

- UI/UX design helps users complete tasks correctly and confidently.
- Forms should collect valid, complete, and necessary data.
- Reports should support specific decisions, not simply display all data.
- Interface and dialogue design define how users move through the system.
- Controls protect reliable input, processing, output, access, audit, and recovery.
- Security must be designed into the interface, workflow, reports, database, and recovery process.

Strong system design connects usability + data quality + security.